Applied Implicit Computational Complexity

Neea Rusch · 15 August 2025

Doctoral Dissertation Defense

I JUST WROTE THE MOST BEAUTIFUL CODE OF MY LIFE.

Inspired by xkcd #664 “Academia vs. Business”

Example

Compare the two programs

Assume variables `X`, `Y`, `Z` $\in \mathbb{Z}$ and `bit` $\in \{0, 1\}$.

```
if (bit) { Z = X }
else     { Z = Y }
```

vs.

```
Z = X * bit + Y * (1 - bit)
```

Static program analysis

Many standard techniques:[1] data flow analysis, type systems, constraint-based analysis, abstract interpretation,…

My “toolbox” comes from **implicit computational complexity**.

[1] See, e.g., Anders Møller and Michael I Schwartzbach. *Static Program Analysis*. Department of Computer Science, Aarhus University, 2024, pp. 1–210. URL: `https://cs.au.dk/~amoeller/spa/spa.pdf` and Flemming Nielson, Hanne R Nielson, and Chris Hankin. *Principles of program analysis*. Springer, 2010, pp. 1–473. ISBN: 9783642084744.

Goals

1. What is implicit computational complexity?
2. How can we *apply* implicit computational complexity?
3. What did we learn from the research?

Applied

Implicit

Computational

Complexity

Applied

Implicit

Computational solvable (in principle) by a computing device

Complexity

Applied

Implicit

Computational solvable (in principle) by a computing device

Complexity at what cost

Applied	
Implicit	in terms of programming languages
Computational	solvable (in principle) by a computing device
Complexity	at what cost

Applied	how to use such theory
Implicit	in terms of programming languages
Computational	solvable (in principle) by a computing device
Complexity	at what cost

Implicit Computational Complexity (ICC)

Let L be a **programming language**, C a **complexity class**, and $[\![p]\!]$ the function computed by program p.

Find a **restriction** $R \subseteq L$, such that the following equality holds:

$$\{[\![p]\!] \mid p \in R\} = C$$

The variables L, C, and R are the parameters that vary greatly between different ICC systems.[2]

[2] Romain Péchoux. "Complexité implicite : bilan et perspectives". Habilitation à Diriger des Recherches (HDR). Université de Lorraine, Dec. 2020. URL: https://hal.univ-lorraine.fr/tel-02978986v3/file/HDR-RP.pdf.

Implicit Computational Complexity (ICC)

Programming language	C, Java, λ-calculus…
+ **restriction**	type system, syntax structure, data flow…
$\Rightarrow$ **complexity class**	PTIME, EXP, L, PP…

Restricting languages is a bit controversial

What are the practical limitations of a non-turing complete language?

Asked 14 years ago Modified 4 months ago Viewed 12k times

74

As there are non-Turing complete languages out there, and given I didn't study Comp Sci at university, could someone explain something that a Turing-incomplete language (like Coq) cannot do?

Practical non-Turing-complete languages?

Asked 15 years, 8 months ago Modified 10 years, 2 months ago Viewed 22k times

55

Nearly all programming languages used are Turing Complete, and while this affords the language to represent any computable algorithm, it also comes with its own set of problems. Seeing as all the algorithms I write are intended to halt, I would like to be able to represent them in a language that guarantees they will halt.

Stackoverflow questions 3492188 and 315340.

Programming languages with ~~restrictions~~ guarantees

(safe) Rust
free of memory errors and data races, with controlled aliasing

Total functional programming
all programs are provably terminating

Interactive theorem proving
require termination, but enable constructing formal proofs

Synchronous programming
for real-time reactive systems with response-time and memory restrictions

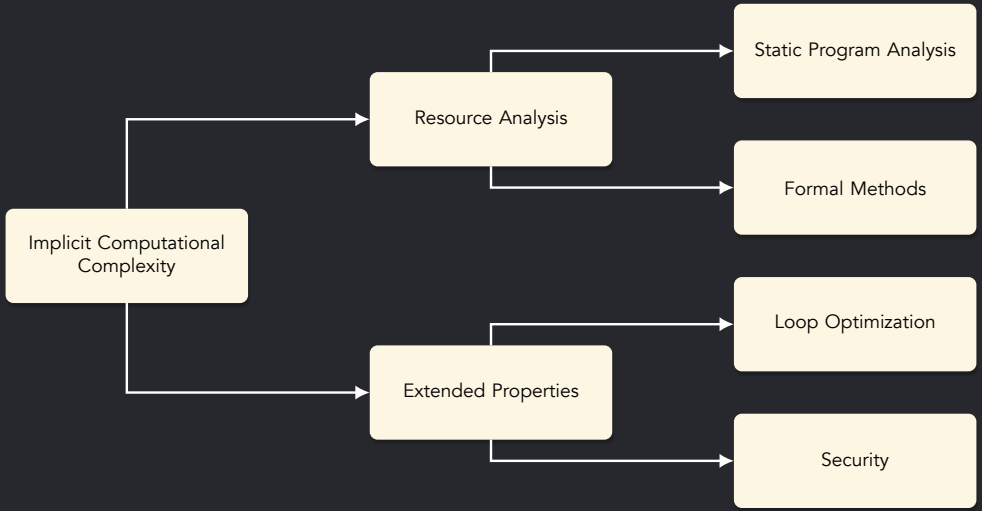
The (Applied) ICC Challenge

Despite compelling features, ICC remains largely a theoretical novelty.
The practical power, limitations, and capabilities are not well-understood.

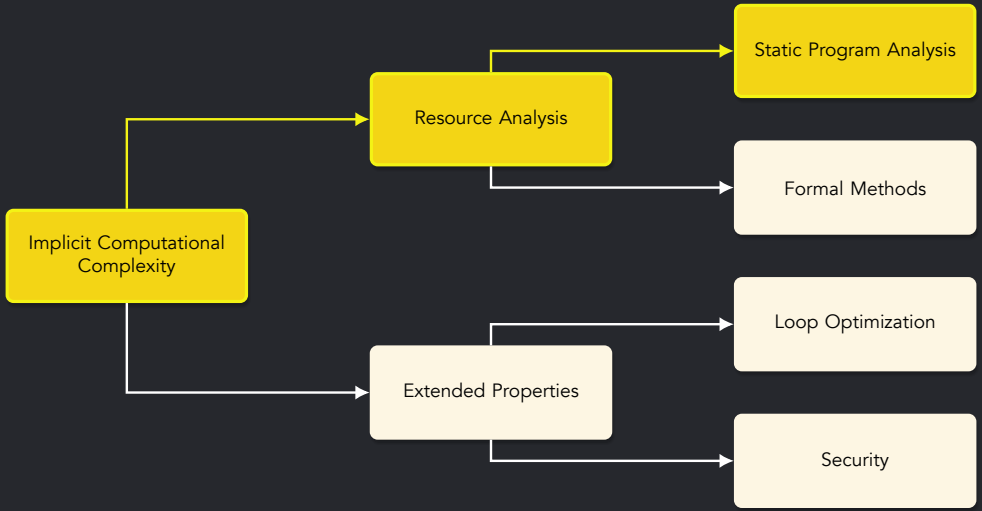
Hypothesis

Implicit computational complexity offers applied utilities when lifted from the theoretical domain.

The Big Picture



The Big Picture



Resource analysis

Apply ICC in designed way toward *automatic* resource analysis.
Some research questions:

- Can we develop practical resource analyses based on ICC theories?
- If theories can be automated, what are their use cases?
- Is the theory correct: can we formally prove, e.g., soundness?

The flow calculus of mwp-bounds[3]

Data flow analysis for certifying that **final values** computed by a deterministic imperative program will be **bounded by polynomials** in the program's **inputs**.

[3] Neil D. Jones and Lars Kristiansen. "A flow calculus of *mwp*-bounds for complexity analysis". In: *ACM Transactions on Computational Logic* 10.4 (Aug. 2009), 28:1–28:41. ISSN: 1557-945X. DOI: 10.1145/1555746.1555752.

The goal is to discover a polynomially bounded data-flow relation for command C, between its variables' initial values x_i and final values x_i': $[\![\texttt{C}]\!](x_i \rightsquigarrow x_i')$.

```
C' ≡ X1 := X2 + X3;
     X1 := X1 + X1
```

$$x_1' \leq 2x_2 + 2x_3$$
$$x_2' \leq x_2$$
$$x_3' \leq x_3$$

✔ *PASS*

```
C'' ≡ X1 := 1;
      loop X2 {X1 := X1 + X1}
```

$$x_1' \leq 2^{x_2}$$
$$x_2' \leq x_2$$

✘ *FAIL*

Mechanics of the flow calculus

1. Imperative programming language

(var) $X_1 \mid X_2 \mid X_3 \mid \ldots$ (aexp) `e + e` | `e * e` (bexp) `e = e` | `e < e` | …

(com) `skip` | `X := e` | `C;C` | `if b then C else C` | `loop X {C}` | `while b do {C}`

- A program is a sequence of commands.
- A program manipulates a fixed number of natural number variables.

Mechanics of the flow calculus

2. Flow coefficients (dependencies)

0 : no dependency m : maximal w : weak polynomial p : polynomial

weaker → *stronger*

```
C' ≡ X1 := X2 + X3;
     X1 := X1 + X1
```

Mechanics of the flow calculus

3. Inference rules

$$\frac{}{\vdash \texttt{xi} : \{^{m}_{\texttt{i}}\}}\ \text{E1} \qquad \frac{\vdash \texttt{xi} : V_1 \quad \vdash \texttt{xj} : V_2}{\vdash \texttt{xi} \star \texttt{xj} : pV_1 \oplus V_2}\ \text{E3} \qquad \frac{\vdash \texttt{e} : V}{\vdash \texttt{xj = e} : 1 \overset{\texttt{j}}{\leftarrow} V}\ \text{A} \quad \ldots$$

- 1 to 3 rules for each expression/command of the imperative language.
- The nondeterminism allows providing guarantees to more programs.

Mechanics of the flow calculus

4. mwp-bound

$$\max(\vec{x}, poly_1(\vec{y})) + poly_2(\vec{z})$$

- The symbols $\vec{x}, \vec{y}, \vec{z}$ are disjoint variable lists.
  Variables characterized by m-flow are in $\vec{x}$, w-flow in $\vec{y}$, and p-flow in $\vec{z}$.
- The $poly_1$ and $poly_2$ are honest polynomials.
  They are built up of variables, constants, and operators + and $\times$.

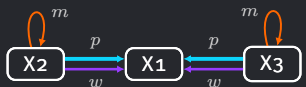
Mechanics of the flow calculus

In summary:

1. Program in imperative programming language (input)
2. Coefficients (track dependence)
3. Inference rules (logic)
4. mwp-bounds (output)

Derivation success

```
C′ ≡ X1 := X2 + X3;
     X1 := X1 + X1
```



$$\dfrac{\dfrac{\dfrac{}{\vdash \texttt{X2} : \begin{pmatrix} 0 \\ m \\ 0 \end{pmatrix}}\,\text{E1} \quad \dfrac{}{\vdash \texttt{X3} : \begin{pmatrix} 0 \\ 0 \\ m \end{pmatrix}}\,\text{E1}}{\vdash \texttt{X2+X3} : \begin{pmatrix} 0 \\ p \\ m \end{pmatrix}}\,\text{E3}}{\vdash \texttt{X1:=X2+X3} : \begin{pmatrix} 0 & 0 & 0 \\ p & m & 0 \\ m & 0 & m \end{pmatrix}}\,\text{A}$$

$$\vdots$$

$$\dfrac{}{\vdash \texttt{X1:=X1+X1} : \begin{pmatrix} p & 0 & 0 \\ 0 & m & 0 \\ 0 & 0 & m \end{pmatrix}}\,\text{A}$$

$$\vdots$$

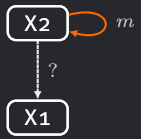
$$\dfrac{}{\vdash \texttt{X1:=X2+X3;X1:=X1+X1} : \begin{pmatrix} 0 & 0 & 0 \\ p & m & 0 \\ p & 0 & m \end{pmatrix}}\,\text{C}$$

$$x_1' \leq x_2 + x_3 \quad \wedge \quad x_2' \leq x_2 \quad \wedge \quad x_3' \leq x_3$$

Derivation failure

$$\forall i, M^*_{ii} = m \quad \frac{\vdash \texttt{c} : M}{\vdash \texttt{loop X}_\ell\ \{\texttt{C}\} : M^* \oplus \{{}^{p}_{\ell}\!\!\rightarrow j \mid \exists i, M^*_{ij} = p\}}\ \mathsf{L}$$

$\texttt{C}'' \equiv$ `X1 := 1;`
`loop X2 {X1 := X1 + X1}`



$$\dfrac{\dfrac{}{\vdash \texttt{X1:=1} : \binom{m}{0}}\ \mathsf{E1}}{\vdots}$$
$$\dfrac{\vdots}{\vdash \texttt{X1:=X1+X1} : \left(\begin{smallmatrix} p & 0 \\ 0 & m \end{smallmatrix}\right)}\ \mathsf{A}$$
$$\vdots$$
$$\bot$$

Derivation __?__

```
if (bit) { Z = X }
else     { Z = Y }
```

vs.

```
Z = X * bit + Y * (1 - bit)
```

Flow calculus application challenges

- How to handle **nondeterminism**?
- How to handle **derivation failure**?

The non-determinism problem

Analysis of expression X2+X3 has 3 derivations.

$$\frac{}{\vdash \mathtt{e} : \{^{w}_{\mathtt{i}} \mid \mathtt{Xi} \in \mathrm{var}(\mathtt{e})\}}\ \text{E2} \qquad \text{by inference rule E2} \qquad \begin{pmatrix} 0 \\ w \\ w \end{pmatrix}$$

$$\frac{\vdash \mathtt{Xi} : V_1 \quad \vdash \mathtt{Xj} : V_2}{\vdash \mathtt{Xi}\star\ \mathtt{Xj} : pV_1 \oplus V_2}\ \text{E3} \qquad \text{by inference rules E1 and E3} \qquad \begin{pmatrix} 0 \\ p \\ m \end{pmatrix}$$

$$\frac{\vdash \mathtt{Xi} : V_1 \quad \vdash \mathtt{Xj} : V_2}{\vdash \mathtt{Xi}\star\ \mathtt{Xj} : V_1 \oplus pV_2}\ \text{E4} \qquad \text{by inference rules E1 and E4} \qquad \begin{pmatrix} 0 \\ m \\ p \end{pmatrix}$$

In general, n choices yields 3^n derivations.

FIX: Handling non-determinism

Idea: track derivation choices as functions from choices to coefficients.

- ▷ If a coefficient depends on a choice: represent as 3 elements.
- ▷ If independent: represent as a single element.

We track not only dependencies, but a history of derivation choices.

Internalized non-determinism

```
X1 = X2 + X3
```

↓

$$\begin{pmatrix} m & 0 & 0 \\ m & m & 0 \\ p & 0 & m \end{pmatrix} \quad \begin{pmatrix} m & 0 & 0 \\ p & m & 0 \\ m & 0 & m \end{pmatrix} \quad \begin{pmatrix} m & 0 & 0 \\ w & m & 0 \\ w & 0 & m \end{pmatrix}$$

↓

$$\begin{pmatrix} m & 0 & 0 \\ m.\delta(0,0)+p.\delta(1,0)+w.\delta(2,0) & m & 0 \\ p.\delta(0,0)+m.\delta(1,0)+w.\delta(2,0) & 0 & m \end{pmatrix}$$

The failure problem

C ≡ `while (b) do {X1:=X2+X2}`

Derivation of `X1:=X2+X2` has two matrices: $\begin{pmatrix} 0 & 0 \\ p & m \end{pmatrix}$ and $\begin{pmatrix} 0 & 0 \\ w & m \end{pmatrix}$.

The inference rule for analyzing an unbounded loop is

$\forall i, M^*_{ii} = m$ and $\forall i, j, M^*_{ij} \neq p$ $\dfrac{\vdash \texttt{C} : M}{\vdash \texttt{while b do \{C\}} : M^*}$ W

Choosing $\begin{pmatrix} 0 & 0 \\ p & m \end{pmatrix}$ fails, but choosing $\begin{pmatrix} 0 & 0 \\ w & m \end{pmatrix}$ succeeds.

FIX: New way to represent failure

Idea: We introduce ∞ flow to represent non-polynomial dependencies.

0 : none m : maximal w : weak polynomial p : polynomial **∞ : failure**

Every derivation can be completed without restarts.
Captures localized information about where failure occurs.
Once failure is introduced, it cannot be erased: $\infty \times^{\infty} 0 = \infty$.

```
C ≡ while (b) do {X1:=X2+X2}
```

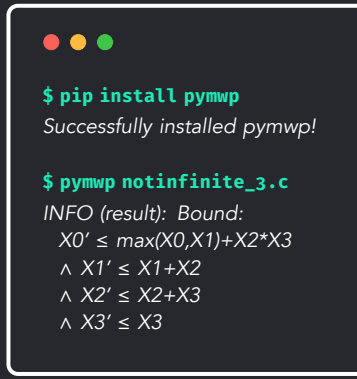
$$\begin{pmatrix} m+\infty\delta(0,0)+\infty\delta(1,0) & 0 \\ \infty\delta(0,0)+\infty\delta(1,0)+w\delta(2,0) & m \end{pmatrix}$$

What did we learn?

The enhanced flow calculus

Q: Can we develop practical resource analyses based on ICC theories?
A: Yes, *eventually*.

- All derivations captured deterministically in one (complex) matrix.
- The mwp-bounds are "hidden" (temporarily).
- The enhanced system gives more information than the original system.



ICC applications

Q: If theories can be automated, what are their use cases?
A: There are *several* applications.

- Automatic static resource analysis, e.g., pymwp[4].
- Specification inference: use obtained bounds in verification.
- The likely pathway toward formal reasoning about complexity.
- …and many more.

[4] Clément Aubert et al. "pymwp: A Static Analyzer Determining Polynomial Growth Bounds". In: *Automated Technology for Verification and Analysis*. Springer Nature Switzerland, 2023, pp. 263–275. ISBN: 9783031453328. DOI: 10.1007/978-3-031-45332-8_14.

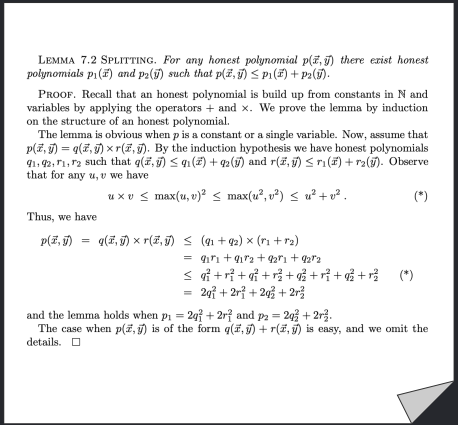
Certifying ICC theories

Q: Is the theory correct, and can we formally prove, e.g., soundness?
A: With high confidence, *likely yes*.

- Added confidence in the correctness of the informal (paper) proofs.
- Evidence of utility to justify the formalization effort.
- Requires making explicit the details of the informal proofs.

The soundness theorem of mwp-calculus. Letting C be a program and M and an mwp-matrix, $\vdash \mathsf{C} : M$ implies $\vDash \mathsf{C} : M$.

Certifying ICC theories



LEMMA 7.2 SPLITTING. *For any honest polynomial $p(\vec{x},\vec{y})$ there exist honest polynomials $p_1(\vec{x})$ and $p_2(\vec{y})$ such that $p(\vec{x},\vec{y}) \leq p_1(\vec{x}) + p_2(\vec{y})$.*

PROOF. Recall that an honest polynomial is build up from constants in N and variables by applying the operators $+$ and $\times$. We prove the lemma by induction on the structure of an honest polynomial.

The lemma is obvious when p is a constant or a single variable. Now, assume that $p(\vec{x},\vec{y}) = q(\vec{x},\vec{y}) \times r(\vec{x},\vec{y})$. By the induction hypothesis we have honest polynomials q_1, q_2, r_1, r_2 such that $q(\vec{x},\vec{y}) \leq q_1(\vec{x}) + q_2(\vec{y})$ and $r(\vec{x},\vec{y}) \leq r_1(\vec{x}) + r_2(\vec{y})$. Observe that for any u, v we have

$$u \times v \;\leq\; \max(u,v)^2 \;\leq\; \max(u^2,v^2) \;\leq\; u^2 + v^2\,. \qquad (*)$$

Thus, we have

$$\begin{aligned} p(\vec{x},\vec{y}) \;&=\; q(\vec{x},\vec{y}) \times r(\vec{x},\vec{y}) \;\leq\; (q_1+q_2)\times(r_1+r_2) \\ &=\; q_1r_1 + q_1r_2 + q_2r_1 + q_2r_2 \\ &\leq\; q_1^2 + r_1^2 + q_1^2 + r_2^2 + q_2^2 + r_1^2 + q_2^2 + r_2^2 \qquad (*) \\ &=\; 2q_1^2 + 2r_1^2 + 2q_2^2 + 2r_2^2 \end{aligned}$$

and the lemma holds when $p_1 = 2q_1^2 + 2r_1^2$ and $p_2 = 2q_2^2 + 2r_2^2$.

The case when $p(\vec{x},\vec{y})$ is of the form $q(\vec{x},\vec{y}) + r(\vec{x},\vec{y})$ is easy, and we omit the details. □

```
Lemma splitting p:
  ∃ p₁ p₂, eval st p ≤ eval st p₁ + eval st p₂.
Proof.
elim: p⇒ [n|x|????|? [q₁[q2 Ht₁]]? [r₁[r₂ Ht₂]]] /=.
- ∃ n, 0; by rewrite addn0 leqnn.
- ∃ x, 0; by rewrite addn0 leqnn.
- eexists; eexists; by [].
- ∃ ((eval st q₁^2).*2 + (eval st r₁^2).*2),
  ((eval st q₂^2).*2 + (eval st r₂^2).*2) ⇒ /=.
  rewrite (leq_trans (leq_mul Ht₁ Ht₂)) // {Ht₁ Ht₂}.
  rewrite !mulnDl !mulnDr -addnA.
  apply: (leq_trans(leq_mulsqn _ _ _ )).
  rewrite addnCA.
  apply: (leq_trans(leq_mulsqn _ _ _ )).
  rewrite addnAC -!addnn -!addnA
          !leq_add2l !addnA leq_add2r.
  apply: (leq_trans(leq_mulsqn _ _ _ )).
  apply: (leq_trans(leq_mulnsq _ _ _ )).
  by rewrite !leq_add2r addnC.
Qed.
```

informal ⟶ *formal*

Analyzing extended properties

Q: Can we use ICC to track *other* properties, beyond resources?
A: Yes, with adjustment.

- Some ICC systems are flexible – can track other properties.
- Complexity guarantee may be lost, but something else is gained.
- Develops deeper understanding of the theory.
- Pushes to introduce ICC to new research communities.

Takeaway messages

Implicit computational complexity

1. offers complementary techniques for resource analysis
2. analyses can be adjusted to track other program properties
3. has potential to guarantee properties by construction

Implicit computational complexity is much broader in utility than the name implies